

Code Verification via the Method of Manufactured Solutions

Marvyn Bailly

November 14, 2024

Motivation

- **Challenges in Numerical Simulations:**

- Numerical codes are complex, can be long, and prone to subtle bugs.
- Identifying these bugs can be time-consuming and difficult.

- **Importance of Verification:**

- Ensuring that the numerical method accurately solves the intended mathematical model.
- Verification provides confidence in the simulation results.

Code Verification

- **Primary Goal:**

- To convincingly demonstrate that the numerical code accurately solves the PDE with the prescribed initial and boundary conditions.

- **Note:**

- Code verification does not inherently debug the code.

Motivation: Verification Difficulties

Difficulties:

- Some numerical methods may exhibit obvious failures.
- Conversely, other methods might appear to perform well visually but still produce inaccurate results.

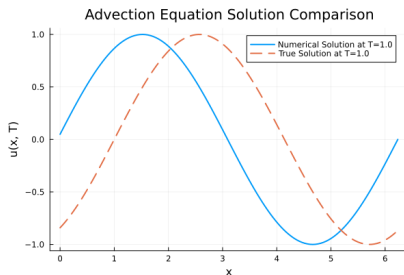
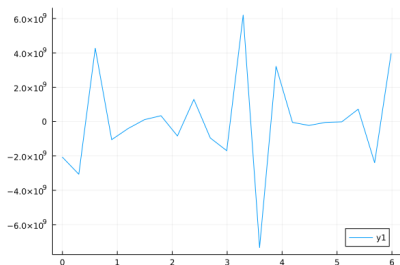


Figure: Left: Method Failing (Blowing Up) Right: Method Appearing Correct

Code Verification: Most Convincing Method

- **Using Known True Solutions:**

- Analytical solve the PDE to find the true solution and compare against the numerical result.

- **Limitations:**

- True solutions are not always available, especially for complex or nonlinear PDEs.

Method of Manufactured Solutions (MMS)

- **Concept Overview:**

- MMS is a technique for code verification when the analytical solution is not unavailable.
- It involves "manufacturing" a solution and modifying the original PDE to incorporate this solution.
- Solve the modified PDE numerically and compare the numerical solution to the manufactured solution.

MMS: Theoretical Framework

- **Original PDE:**

$$\mathcal{L}(u(t, \mathbf{x})) = 0$$

where $\mathcal{L}$ is a differential operator representing the PDE.

- **Manufactured Solution:**

$$U(t, \mathbf{x})$$

A user-defined solution that may not satisfy the original PDE.

- **Determining the Source Term:**

$$\mathcal{L}(U(t, \mathbf{x})) = Q(t, \mathbf{x})$$

By substituting $U(t, \mathbf{x})$ into the PDE, we derive the source term $Q(t, \mathbf{x})$.

- **Modified PDE:**

$$\mathcal{L}(u(t, \mathbf{x})) = Q(t, \mathbf{x})$$

Now, $U(t, \mathbf{x})$ is an exact solution to the modified PDE.

Example: Advection Equation

1D Advection Equation:

$$\mathcal{L}(u) \equiv u_t + au_x = 0 \quad (1)$$

- **Manufactured Solution:**

$$U(t, x) = \cos(t) + \sin(x)$$

- **Source Term:**

$$\mathcal{L}(U) \equiv -\sin(t) + a \cos(x) = Q(t, x)$$

Modified Advection Equation

$$\mathcal{L}(u) = Q(t, x) \quad \text{or} \quad u_t + au_x = -\sin(t) + a \cos(x)$$

Guidelines for Creating Manufactured Solutions

- **Generality:**

- Ensure the manufactured solution is sufficiently general to exercise all terms in the governing PDE.
- E.g., Equation 1 requires a solution with x and t dependence.

- **Non-Triviality:**

- The solution should involve non-trivial derivatives to ensure all terms in the PDE are active.
- E.g., Linear solutions will not suffice for verifying 2nd-order PDEs.

- **Compliance with PDE Assumptions:**

- The solution must adhere to the inherent assumptions of the PDE.
- E.g., if the PDE is defined for positive times, the solution should be valid for $t > 0$.

- **Physical Realism:**

- The manufactured solution does not need to be physically realistic.

Initial and Boundary Conditions

- **Initial Conditions:**

- The manufactured solution does not need to satisfy the original initial condition for the PDE.

- **Boundary Conditions:**

- The boundary conditions can be challenging.
- Design different manufactured solutions to test different types of boundary conditions.

- **Current Implementation**

- Requires initial condition for manufactured solution to match original initial condition.
- Requires boundary conditions to be periodic.

Example: Advection Equation

Initial Condition

$$u(0, x) = f(x) = \sin(x)$$

- **Manufactured Solution:**

$$\tilde{U}(t, x) = \cos(t) + \sin(x)$$

but

$$\tilde{U}(0, x) = 1 + \sin(x) \neq f(x)$$

so take

$$U(t, x) = -1 + \cos(t) + \sin(x)$$

Applying MMS for Code Verification

- **Verification Process:**

- Set up the modified PDE for numerical solving.
- Run the numerical solver to obtain the numerical solution $v(t, \mathbf{x})$.
- Compare $v(t, \mathbf{x})$ with the manufactured solution $U(t, \mathbf{x})$.

- **Order of Accuracy:**

- Evaluate whether the numerical method achieves the expected order of accuracy when refining the mesh.
- Consistency in the order of accuracy between the original and modified PDEs confirms the correctness of the numerical implementation.

Reminder: What is Accuracy?

Accuracy in Numerical Methods:

- Accuracy measures how closely the numerical solution approximates the exact solution.
- For a method with order of accuracy p , the global error E typically scales with the grid spacing h as:

$$E \propto h^p$$

- **Higher Order = Greater Accuracy:** As p increases, the numerical solution converges to the exact solution more quickly as the grid is refined.
- **Practical Meaning:** For example, a second-order method (with $p = 2$) will reduce its error by roughly a factor of 4 when the grid spacing h is halved.

Approximating Accuracy

- **Procedure:**

- ① Conduct simulations using two different grid resolutions: h and h/r , where r is the refinement ratio.
- ② Compute the global errors E_1 and E_2 corresponding to each grid size respectively.
- ③ For a p th-order method, we expect

$$E_1 \approx C_1 h^p \text{ and } E_2 \approx C_2 \left(\frac{h}{r}\right)^p$$

- ④ Calculate the observed order of accuracy p

$$p \approx \frac{\log\left(\frac{E_1}{E_2}\right)}{\log r}$$

- Note: This is a rough approximation.

Implementation

Handling the Source Term:

- The source term $Q(x, t)$ can become quite complex depending on the PDE and manufactured solution.
- To simplify this process:
 - Solve for $Q(x, t)$ symbolically.
 - Convert symbolic $Q(x, t)$ into a forcing term for numerical solver.

Language Choice:

- This approach requires a language capable of both symbolic manipulation, code generation and code execution.

Forcing Terms Can Get Complex

Example of a Complex Forcing Term:

$$\begin{aligned} \text{out}[33] = & \frac{1}{3} \left(3 \text{et0} \cos \theta \cos [\cos \theta t + x^2 + y^2] \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) - \right. \\ & 3 \text{et0} \cos \theta \sin [\cos \theta t + x^2 + y^2] \left(\frac{3}{2} + \sin [\cos \theta t + x^2 + y^2] \right) + \\ & 6 v_0 x \sin [\cos \theta t + x^2 + y^2] (2 u_0 y \cos [\cos \theta t + x^2 + y^2] - 2 v_0 x \sin [\cos \theta t + x^2 + y^2]) - 12 u_0 \\ & x y \cos [\cos \theta t + x^2 + y^2] (u_0 y \cos [\cos \theta t + x^2 + y^2] - v_0 x \sin [\cos \theta t + x^2 + y^2]) + 8 u_0 x y \cos [\cos \theta t + x^2 + y^2] \\ & (-u_0 + v_0 x y) \cos [\cos \theta t + x^2 + y^2] + 2 u_0 x^2 \sin [\cos \theta t + x^2 + y^2] - \\ & 8 u_0 x y \cos [\cos \theta t + x^2 + y^2] (2 u_0 x \cos [\cos \theta t + x^2 + y^2] + v_0 y \sin [\cos \theta t + x^2 + y^2]) - \\ & 8 v_0 x y \sin [\cos \theta t + x^2 + y^2] (u_0 x \cos [\cos \theta t + x^2 + y^2] + 2 v_0 y \sin [\cos \theta t + x^2 + y^2]) + \\ & 6 u_0 x y (\cos [\cos \theta t + x^2 + y^2] \\ & (-u_0 - 2 v_0 x y) \cos [\cos \theta t + x^2 + y^2] + 2 u_0 y^2 \sin [\cos \theta t + x^2 + y^2]) + 8 v_0 x y (\cos [\cos \theta t + x^2 + y^2] \\ & + 6 v_0 x y (\cos [\cos \theta t + x^2 + y^2] \\ & (2 v_0 x^2 \cos [\cos \theta t + x^2 + y^2] + (v_0 - 2 u_0 x y) \sin [\cos \theta t + x^2 + y^2]) + \\ & \frac{1}{\text{gamR}} (3 (-1 + \text{gam}) \text{tCK} (4 (u_0^2 - v_0^2) x^2 \cos [2 (\cos \theta t + x^2 + y^2)] + \\ & 2 (\text{et0} - \text{eps} v_0^2 - 2 \text{eps} u_0^2 x^2) \sin [\cos \theta t + x^2 + y^2] + \\ & 2 \cos [\cos \theta t + x^2 + y^2] (2 \text{et0} x^2 + \text{eps} (u_0^2 - 2 v_0^2 x^2) + (u_0^2 - v_0^2) \sin [\cos \theta t + x^2 + y^2])) + \\ & \frac{1}{\text{gamR}} (3 (-1 + \text{gam}) \text{tCK} (4 (u_0^2 - v_0^2) y^2 \cos [2 (\cos \theta t + x^2 + y^2)] + \\ & 2 (\text{et0} - \text{eps} v_0^2 - 2 \text{eps} u_0^2 y^2) \sin [\cos \theta t + x^2 + y^2] + \\ & 2 \cos [\cos \theta t + x^2 + y^2] (2 \text{et0} y^2 + \text{eps} (u_0^2 - 2 v_0^2 y^2) + (u_0^2 - v_0^2) \sin [\cos \theta t + x^2 + y^2])) + \\ & 6 \theta \cos \theta u_0 x \cos [\cos \theta t + x^2 + y^2] \left(\frac{3}{2} + \sin [\cos \theta t + x^2 + y^2] \right) \\ & \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) + (-1 + \text{gam}) \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) + \right. \right. \\ & \left. \left. \frac{1}{2} (-v_0^2 (\cos [\cos \theta t + x^2 + y^2] + (-1 + \text{gam}) \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) \right) - \right. \right. \right. \\ & \left. \left. \left. 6 \theta \cos \theta v_0 y \sin [\cos \theta t + x^2 + y^2] \left(\frac{3}{2} + \sin [\cos \theta t + x^2 + y^2] \right) \right) \right) \right. \\ & \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) + (-1 + \text{gam}) \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) + \right. \right. \\ & \left. \left. \frac{1}{2} (-v_0^2 (\cos [\cos \theta t + x^2 + y^2] + (-1 + \text{gam}) \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) \right) - \right. \right. \right. \\ & \left. \left. \left. 3 \theta \cos \theta v_0 y (\cos [\cos \theta t + x^2 + y^2] (\text{et0} \cos [\cos \theta t + x^2 + y^2] (3 + 2 \cos [\cos \theta t + x^2 + y^2]) - \right. \right. \right. \\ & \left. \left. \left. \text{et0} \sin [\cos \theta t + x^2 + y^2] (3 + 2 \sin [\cos \theta t + x^2 + y^2]) + \right. \right. \right. \\ & \left. \left. \left. 2 (-1 + \text{gam}) \left(\frac{3}{2} + \sin [\cos \theta t + x^2 + y^2] \right) ((-\text{et0} - \text{eps} v_0^2) \sin [\cos \theta t + x^2 + y^2] - \right. \right. \right. \\ & \left. \left. \left. \cos [\cos \theta t + x^2 + y^2] (\text{eps} u_0^2 + (u_0^2 - v_0^2) \sin [\cos \theta t + x^2 + y^2]) + \right. \right. \right. \\ & \left. \left. \left. 2 (-1 + \text{gam}) \cos [\cos \theta t + x^2 + y^2] \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) + \right. \right. \right. \\ & \left. \left. \left. \frac{1}{2} (-v_0^2 (\cos [\cos \theta t + x^2 + y^2] + (-1 + \text{gam}) \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) \right) - \right. \right. \right. \\ & \left. \left. \left. 3 \theta \cos \theta u_0 x (\cos [\cos \theta t + x^2 + y^2] (\text{et0} \cos [\cos \theta t + x^2 + y^2] (3 + 2 \cos [\cos \theta t + x^2 + y^2]) - \right. \right. \right. \\ & \left. \left. \left. \text{et0} \sin [\cos \theta t + x^2 + y^2] (3 + 2 \sin [\cos \theta t + x^2 + y^2]) + \right. \right. \right. \\ & \left. \left. \left. 2 (-1 + \text{gam}) \left(\frac{3}{2} + \sin [\cos \theta t + x^2 + y^2] \right) ((-\text{et0} - \text{eps} v_0^2) \sin [\cos \theta t + x^2 + y^2] - \right. \right. \right. \\ & \left. \left. \left. \cos [\cos \theta t + x^2 + y^2] (\text{eps} u_0^2 + (u_0^2 - v_0^2) \sin [\cos \theta t + x^2 + y^2]) + \right. \right. \right. \\ & \left. \left. \left. 2 (-1 + \text{gam}) \cos [\cos \theta t + x^2 + y^2] \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) + \right. \right. \right. \\ & \left. \left. \left. \frac{1}{2} (-v_0^2 (\cos [\cos \theta t + x^2 + y^2] + (-1 + \text{gam}) \left(\text{et0} \left(\frac{3}{2} + \cos [\cos \theta t + x^2 + y^2] \right) \right) - \right. \right. \right. \right. \end{aligned}$$

Figure: Source term for 2-D laminar compressible Navier-Stokes equations

Python vs Julia

- **Symbolic Computation:**

- Both Python and Julia offer symbolic computation capabilities through their respective libraries, SymPy and Symbolics.jl.
- Symbolics.jl can export expressions as optimized C++.

- **Pros of Julia:**

- Julia compiles code using Just-in-Time compilation, allowing it to achieve speeds comparable to languages like C.
- Julia is designed with numerical and scientific computing in mind.
- Julia offers built-in support for parallel and distributed computing.

- **Cons of Julia:**

- Julia's library ecosystem is limited compared to Python's.
- Julia's community and available resources are smaller compared to Python

Implementation: Pseudocode

Procedure:

- ① User specifies the PDE and a manufactured solution $U(x, t)$ in Julia.
 - ② User specifies where to place forcing term using comments
 - ③ Symbolically solves for the source term $Q(x, t) = \mathcal{L}(U(x, t))$
 - ④ $Q(x, t)$ is written to the numerical solver as a forcing term
 - ⑤ Run the numerical solver for two different mesh sizes
 - ⑥ Approximate order of accuracy
- *Let's see it in action!*

Code and Simple Example

Numerical Test Setup:

- **Equation:** 1D advection equation $\mathcal{L}(u) \equiv u_t + 4u_x = 0$
- **Initial Condition:** $u(x, 0) = \sin(x)$.
- **Boundary Conditions:** 2π -periodic.
- **Manufactured Solution:** $U(x, t) = e^{xt} \cos(t) \sin(x)$
- **Numerical Method:** Lax-Wendroff scheme (second order)

Oceananigans Example

PDE: Diffusion Equation

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2} + f(x, t),$$

subject to

$$u(x, 0) = e^{\frac{-x^2}{2w^2}}$$

where:

- $u(x, t)$: Scalar field (temperature).
- κ : Diffusion coefficient.
- w : Width of Guassian.
- $f(x, t)$: Forcing term derived from the manufactured solution.

Future Plans

- **Expand Boundary Condition Types:**
 - Incorporate various boundary conditions such as Dirichlet and Neumann to test the robustness of numerical methods.
- **Enhance Code Verification:**
 - Implement verification between Julia and C++ code.
- **Increase PDE Complexity:**
 - Apply MMS to more complex PDEs, aiming to verify the implementation of Navier-Stokes equations!
- **Generalize Manufactured Solutions:**
 - Allow users to define manufactured solutions on diverse subsets of 2-D and 3-D space.
- **Make a Library:**
 - Create a MMS library for users to download for general verification.

References I



P. J. Roache.

Code Verification by the Method of Manufactured Solutions.

Journal of Fluids Engineering - Transactions of the ASME, 124:4–10, 2002.

Available:

<https://api.semanticscholar.org/CorpusID:121230632>



Symbolics.jl Documentation.

<https://symbolics.juliasymbolics.org/>